

Kubernetes-06

Tasks:

1. Create a ConfigMap from a directory containing multiple files and inject the variables into a pod as environment variables.
2. Create a ConfigMap from a file and mount it as a volume inside a pod, ensuring the configuration data is available as files.
3. Create a Secret with sensitive information (username and password) and inject it into a pod as environment variables.
4. Create a Secret using a YAML file, mount it as a volume in a pod, and verify the specific Secret values are available as files.
5. Inject a ConfigMap as environment variables and a Secret as files into the same pod, ensuring both are accessible within the pod.

Services (Kubernetes Objects) We Are Using

ConfigMap

—> Used to store **non-sensitive configuration data**

Examples:

- App mode
- Port number
- Color
- Environment variables

It can be used in:

- Environment variables
- Mounted as files (volume)

Secret

—> Used to store **sensitive data**

Examples:

- Database username
- Password
- API keys
- Tokens

It is:

- Base64 encoded
- More secure than ConfigMap

It can be used in:

- Environment variables
- Mounted as files (volume)

TASK 1: Create a ConfigMap from a directory containing multiple files and inject the variables into a pod as environment variables.

Create ConfigMap from a directory & inject as Environment Variables

Step 1: Created directory

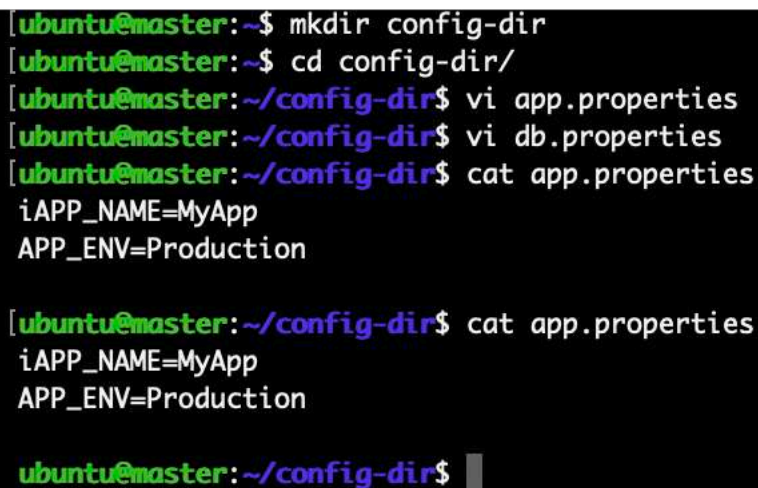
```
config-dir/  
├── app.properties  
└── db.properties
```

Example content:

app.properties

```
APP_NAME=MyApp  
APP_ENV=Production  
db.properties
```

```
DB_HOST=mysql-service  
DB_USER=admin  
DB_PASS=secret123
```



```
[ubuntu@master:~]$ mkdir config-dir  
[ubuntu@master:~]$ cd config-dir/  
[ubuntu@master:~/config-dir]$ vi app.properties  
[ubuntu@master:~/config-dir]$ vi db.properties  
[ubuntu@master:~/config-dir]$ cat app.properties  
iAPP_NAME=MyApp  
APP_ENV=Production  
  
[ubuntu@master:~/config-dir]$ cat app.properties  
iAPP_NAME=MyApp  
APP_ENV=Production  
  
ubuntu@master:~/config-dir$
```

By mistake I have written iAPP_NAME
I removed { i } from file

Step 2: Create ConfigMap from Directory

Come out of the directory first:

```
cd ..
```

Now run:

```
kubectl create configmap my-config --from-file=config-dir/
```

```
[ubuntu@master:~/config-dir]$ cd
ubuntu@master:~$ kubectl create configmap my-config --from-file=config-dir/
configmap/my-config created
ubuntu@master:~$ kubectl get configmap
NAME          DATA   AGE
kube-root-ca.crt  1      6d21h
my-config      2       8s
```

Verify ConfigMap

```
kubectl get configmap
```

```
kubectl describe configmap my-config
```

```
ubuntu@master:~$ kubectl describe configmap my-config
```

```
Name:      my-config
Namespace:  default
Labels:     <none>
Annotations: <none>
```

```
Data
```

```
====
```

```
app.properties:
```

```
----
```

```
APP_NAME=MyApp
```

```
APP_ENV=Production
```

```
db.properties:
```

```
----
```

```
DB_HOST=mysql-service
```

```
DB_USER=admin
```

```
DB_PASS=secret123
```

```
BinaryData
```

```
====
```

```
Events: <none>
```

```
[ubuntu@master:~$ kubectl get configmap
```

NAME	DATA	AGE
kube-root-ca.crt	1	6d21h
my-config	2	27s

```
ubuntu@master:~$
```

Next Step: Inject ConfigMap into a Pod as Environment Variables

Step 3: Create Pod YAML

Create file:

```
vi pod.yaml
```

```
ubuntu@master:~$ cat pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: config-env-pod
spec:
  containers:
  - name: nginx-container
    image: nginx
    envFrom:
    - configMapRef:
        name: my-config

ubuntu@master:~$
```

Step 4: Apply Pod

kubectl apply -f pod.yaml
Check pod status:

kubectl get pods
Wait until STATUS = Running.

```
ubuntu@master:~$ kubectl apply -f pod.yaml
pod/config-env-pod created
ubuntu@master:~$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
config-env-pod 1/1     Running   0           6s
ubuntu@master:~$
```

Step 3: Verify Environment Variables Inside Pod

Now check if values are injected:

```
kubectl exec -it config-env-pod -- printenv | grep APP
```

```
kubectl exec -it config-env-pod -- printenv | grep DB
```

```
config-env-pod 1/1 Running 0s
ubuntu@master:~$ kubectl exec -it config-env-pod -- printenv | grep APP
app.properties=APP_NAME=MyApp
APP_ENV=Production
ubuntu@master:~$ kubectl exec -it config-env-pod -- printenv | grep DB
db.properties=DB_HOST=mysql-service
DB_USER=admin
DB_PASS=secret123
ubuntu@master:~$
```

What happened here?

- Each file inside ConfigMap became a key
- Since file content is KEY=VALUE format
- envFrom injected them as environment variables

Step 4: Check environment variables inside the Pod

```
kubectl exec -it config-env-pod -- printenv
```

```
ubuntu@master:~$ kubectl exec -it config-env-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=config-env-pod
NGINX_VERSION=1.29.5
NJS_VERSION=0.9.5
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
db.properties=DB_HOST=mysql-service
DB_USER=admin
DB_PASS=secret123
```

```
app.properties=APP_NAME=MyApp
APP_ENV=Production
```

```
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
TERM=xterm
HOME=/root
ubuntu@master:~$
```

Login into the container shell:

```
kubectl exec -it config-env-pod -- /bin/bash
```

Then inside container:

Env

```
HOME=/root
ubuntu@master:~$ kubectl exec -it config-env-pod -- /bin/bash
[root@config-env-pod:/# env
db.properties=DB_HOST=mysql-service
DB_USER=admin
DB_PASS=secret123

app.properties=APP_NAME=MyApp
APP_ENV=Production

KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_SERVICE_PORT=443
HOSTNAME=config-env-pod
PWD=/
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
HOME=/root
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
DYNPKG_RELEASE=1~trixie
NJS_VERSION=0.9.5
TERM=xterm
SHLVL=1
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PORT=443
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NGINX_VERSION=1.29.5
NJS_RELEASE=1~trixie
_=/usr/bin/env
root@config-env-pod:/#
```

TASK 2 : Create a ConfigMap from a file and mount it as a volume inside a pod, ensuring the configuration data is available as files.

Create ConfigMap from a file & mount it as a Volume inside a Pod

This is how real applications usually consume config — as files.

Goal

Create a ConfigMap from a single file and mount it inside a Pod so the configuration is available as a file.

Step 1: Create a Config File

Create a new file:

Vi app-config.txt:

A terminal window with a black background and green text. The prompt is [ubuntu@master:~\$. The first command is vi app-config.txt. The second command is cat app-config.txt. The output of cat is APP_NAME=MyVolumeApp, APP_ENV=Development, DB_HOST=mysql, and DB_USER=root. The prompt returns to [ubuntu@master:~\$.

```
[ubuntu@master:~$ vi app-config.txt
[ubuntu@master:~$ cat app-config.txt
APP_NAME=MyVolumeApp
APP_ENV=Development
DB_HOST=mysql
DB_USER=root

ubuntu@master:~$
```

Step 2: Create a ConfigMap from the File

```
kubectl create configmap app-config --from-file=app-config.txt
```

Step 3: Verify the ConfigMap

```
kubectl describe configmap app-config
```

```
ubuntu@master:~$ kubectl create configmap app-config --from-file=app-config.txt
configmap/app-config created
ubuntu@master:~$ kubectl describe configmap app-config
Name:         app-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
app-config.txt:
----
APP_NAME=MyVolumeApp
APP_ENV=Development
DB_HOST=mysql
DB_USER=root

BinaryData
=====

Events: <none>
[ubuntu@master:~$ kubectl get configmap
NAME          DATA  AGE
app-config    1      38s
kube-root-ca.crt 1      6d22h
my-config     2      22m
ubuntu@master:~$
```

Step 4: Create a Pod that Mounts the ConfigMap as a Volume

Create a YAML file named `pod-configmap-volume.yaml`

```
[ubuntu@master:~]$ vi pod-configmap-volume.yaml
[ubuntu@master:~]$ cat pod
cat: pod: No such file or directory
[ubuntu@master:~]$ cat pod-configmap-volume.yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: ["/bin/sh", "-c", "sleep 3600"]
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
  volumes:
  - name: config-volume
    configMap:
      name: app-config

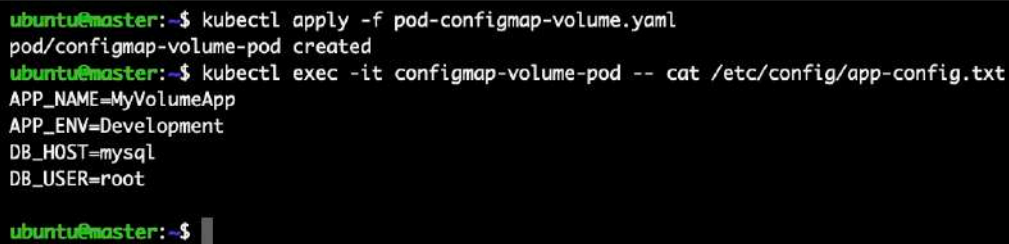
ubuntu@master:~$
```

Step 5: Deploy the Pod

`kubectl apply -f pod-configmap-volume.yaml`

Step 6: Verify the Mounted File Inside the Pod

```
kubectl exec -it configmap-volume-pod -- cat /etc/config/app-config.txt
```



```
ubuntu@master:~$ kubectl apply -f pod-configmap-volume.yaml
pod/configmap-volume-pod created
ubuntu@master:~$ kubectl exec -it configmap-volume-pod -- cat /etc/config/app-config.txt
APP_NAME=MyVolumeApp
APP_ENV=Development
DB_HOST=mysql
DB_USER=root
ubuntu@master:~$
```

How It Works

- The ConfigMap contains one key: `app-config.txt`
- Kubernetes mounts that key as a file
- The file appears inside the container at `/etc/config/app-config.txt`
- The application can read it like a normal file

TASK 3: Create a Secret with sensitive information (username and password) and inject it into a pod as environment variables.

Create a Secret with sensitive information (username and password) and inject it into a Pod as environment variables

Goal

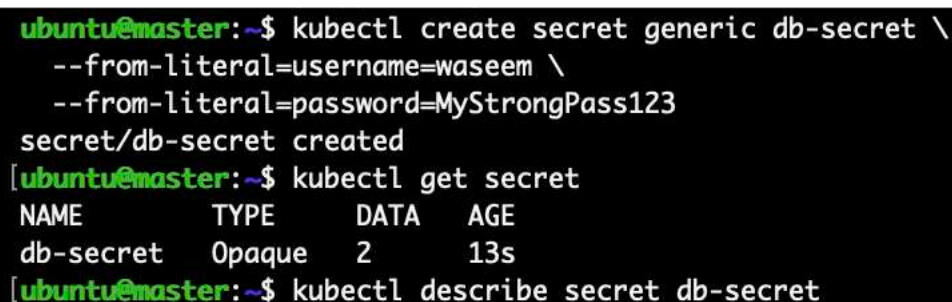
Create a Kubernetes Secret containing:

- Username: **waseem**
- Password: **MyStrongPass123**

Then inject them into a Pod as environment variables.

Step 1: Create the Secret

```
kubectl create secret generic db-secret \
  --from-literal=username=waseem \
  --from-literal=password=MyStrongPass123
```



```
ubuntu@master:~$ kubectl create secret generic db-secret \
  --from-literal=username=waseem \
  --from-literal=password=MyStrongPass123
secret/db-secret created
[ubuntu@master:~$ kubectl get secret
NAME          TYPE          DATA   AGE
db-secret     Opaque        2       13s
[ubuntu@master:~$ kubectl describe secret db-secret
```

Step 2: Verify the Secret

```
kubectl get secret
```

```
kubectl describe secret db-secret
```

To see Base64 encoded values:

```
kubectl get secret db-secret -o yaml
```

```
[ubuntu@master:~]$ kubectl get secret
NAME          TYPE      DATA   AGE
db-secret     Opaque    2        13s
[ubuntu@master:~]$ kubectl describe secret db-secret
Name:         db-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type: Opaque

Data
====
password: 15 bytes
username: 6 bytes
[ubuntu@master:~]$ kubectl get secret db-secret -o yaml
apiVersion: v1
data:
  password: TXlTdHJvbmdQYXNzMTIz
  username: d2FzZWVt
kind: Secret
metadata:
  creationTimestamp: "2026-02-23T10:02:24Z"
  name: db-secret
  namespace: default
  resourceVersion: "58579"
  uid: b7c73fa5-f9e5-45af-a872-73c9a998c6ae
type: Opaque
[ubuntu@master:~]$
```

Step 3: Create a Pod that Uses the Secret as Environment Variables

Create a file named `pod-secret-env.yaml`

```
[ubuntu@master:~$ vi pod-secret-env.yaml
[ubuntu@master:~$ cat pod-secret-env.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: ["/bin/sh", "-c", "sleep 3600"]
    env:
    - name: DB_USERNAME
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: username
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password

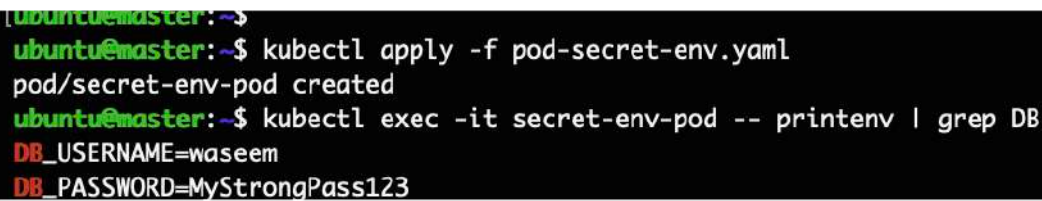
ubuntu@master:~$
```

Step 4: Deploy the Pod

`kubectl apply -f pod-secret-env.yaml`

Step 5: Verify Inside the Pod

```
kubectl exec -it secret-env-pod -- printenv | grep DB
```



```
ubuntu@master:~$  
ubuntu@master:~$ kubectl apply -f pod-secret-env.yaml  
pod/secret-env-pod created  
ubuntu@master:~$ kubectl exec -it secret-env-pod -- printenv | grep DB  
DB_USERNAME=waseem  
DB_PASSWORD=MyStrongPass123
```

How It Works

- The Secret stores sensitive data.
- Kubernetes injects specific keys as environment variables.
- The values are securely referenced inside the Pod.

TASK 4: Create a Secret using a YAML file, mount it as a volume in a pod, and verify the specific Secret values are available as files.

Create a Secret using a YAML file, mount it as a volume in a Pod, and verify the Secret values are available as files

Goal

- Create a Secret using YAML
- Mount it as a volume
- Verify secret values appear as files inside the container

Step 1: Encode the Secret Values in Base64

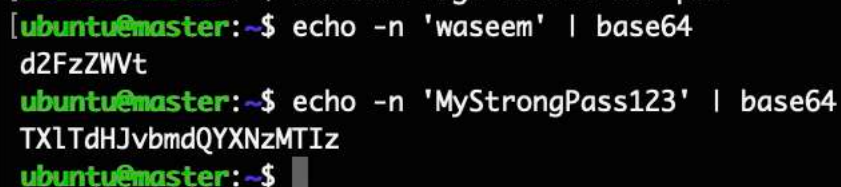
Kubernetes Secrets require Base64 encoding.

Encode username:

```
echo -n 'waseem' | base64
```

Encode password:

```
echo -n 'MyStrongPass123' | base64
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The first command is 'echo -n 'waseem' | base64' and the output is 'd2FzZWVt'. The second command is 'echo -n 'MyStrongPass123' | base64' and the output is 'TXlTdHJvbmdQYXNzMTIz'. The prompt is 'ubuntu@master:~\$' followed by a cursor.

```
[ubuntu@master:~$ echo -n 'waseem' | base64
d2FzZWVt
ubuntu@master:~$ echo -n 'MyStrongPass123' | base64
TXlTdHJvbmdQYXNzMTIz
ubuntu@master:~$
```

Step 2: Create Secret YAML File

Create file:

```
vi secret-volume.yaml
```

```
TXlTdHJvbmdQYXNzMTIz
ubuntu@master:~$ vi secret-volume.yaml
[ubuntu@master:~$ cat secret-volume.yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret-yaml
type: Opaque
data:
  username: d2FzZWVt
  password: TXlTdHJvbmdQYXNzMTIz

ubuntu@master:~$
```

Step 3: Apply the Secret

```
kubectl apply -f secret-volume.yaml
```

Verify:

```
kubectl describe secret db-secret-yaml
```

```
ubuntu@master:~$ kubectl apply -f secret-volume.yaml
secret/db-secret-yaml created
ubuntu@master:~$ kubectl describe secret db-secret-yaml
Name:          db-secret-yaml
Namespace:     default
Labels:        <none>
Annotations:   <none>

Type:  Opaque

Data
====
password:  15 bytes
username:  6 bytes
```

Step 4: Create Pod to Mount Secret as Volume

Create:

```
vi pod-secret-volume.yaml
```

Paste:

```
ubuntu@master:~$ vi pod-secret-volume.yaml
[ubuntu@master:~$ cat pod-secret-volume.yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-volume-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: ["/bin/sh", "-c", "sleep 3600"]
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secret
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: db-secret-yaml
```

Apply:

```
kubectl apply -f pod-secret-volume.yaml
```

```
ubuntu@master:~$ kubectl apply -f pod-secret-volume.yaml
pod/secret-volume-pod created
ubuntu@master:~$ kubectl exec -it secret-volume-pod -- ls /etc/secret
password username
```

Step 5: Verify Inside Pod

List files:

```
kubectl exec -it secret-volume-pod -- ls /etc/secret
```

Expected:

username

password

Check content:

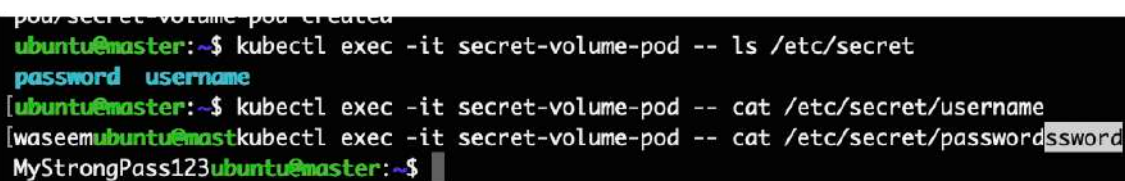
```
kubectl exec -it secret-volume-pod -- cat /etc/secret/  
username
```

```
kubectl exec -it secret-volume-pod -- cat /etc/secret/  
password
```

Expected:

waseem

MyStrongPass123



```
pod/secret-volume-pod created  
ubuntu@master:~$ kubectl exec -it secret-volume-pod -- ls /etc/secret  
password username  
[ubuntu@master:~$ kubectl exec -it secret-volume-pod -- cat /etc/secret/username  
[waseem@ubuntu@master:kubectl exec -it secret-volume-pod -- cat /etc/secret/password]ssword  
MyStrongPass123ubuntu@master:~$
```

How It Works

- Secret stores Base64 encoded data
- Kubernetes decodes it automatically
- Each key becomes a file
- File name = key name

- File content = decoded value

TASK 5 :Inject a ConfigMap as environment variables and a Secret as files into the same pod, ensuring both are accessible within the pod.

Inject a ConfigMap as Environment Variables and a Secret as Files into the Same Pod

Goal

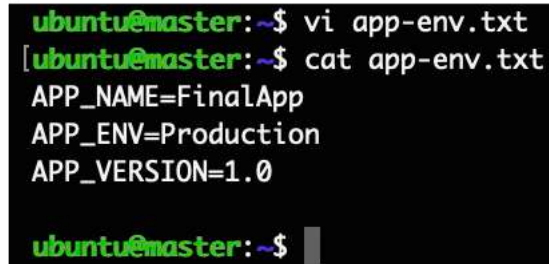
- ConfigMap → Inject as **Environment Variables**
- Secret → Mount as **Files**
- Both must be accessible inside the same Pod

This is real production-level configuration

Step 1: Create Config File

Open file:

```
vi app-env.txt
```

A terminal window with a black background and green text. The prompt is 'ubuntu@master:~\$'. The first command is 'vi app-env.txt'. The second command is '[ubuntu@master:~\$ cat app-env.txt'. The output of the cat command is 'APP_NAME=FinalApp', 'APP_ENV=Production', and 'APP_VERSION=1.0'. The prompt returns to 'ubuntu@master:~\$' with a cursor.

```
ubuntu@master:~$ vi app-env.txt
[ubuntu@master:~$ cat app-env.txt
APP_NAME=FinalApp
APP_ENV=Production
APP_VERSION=1.0

ubuntu@master:~$
```

Step 2: Create ConfigMap from the File

```
kubectl create configmap final-config --from-env-file=app-
env.txt
```

Step 3: Verify ConfigMap

```
kubectl describe configmap final-config
```

```

ubuntu@master:~$ kubectl create configmap final-config --from-env-file=app-env.txt
configmap/final-config created
[ubuntu@master:~$ kubectl get configmap
NAME          DATA   AGE
app-config    1       32m
final-config   3       21s
kube-root-ca.crt 1       6d22h
my-config     2       54m
ubuntu@master:~$ kubectl describe configmap final-config
Name:         final-config
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_VERSION:
----
1.0
APP_ENV:
----
Production
APP_NAME:
----
FinalApp

BinaryData
====

Events: <none>
ubuntu@master:~$

```

Now your ConfigMap is created

—> we already have a secret db use that secret

We will use:

- ConfigMap → final-config
- Secret → db-secret-yaml

```

[ubuntu@master:~$ kubectl get secret
NAME          TYPE      DATA   AGE
db-secret     Opaque    2       26m
db-secret-yaml Opaque    2       14m
ubuntu@master:~$

```

Step 4: Create Pod YAML Using **vi**

Open file:

```
vi combined-pod.yaml
```

```
ubuntu@master:~$ vi combined-pod.yaml
[ubuntu@master:~$ cat combined-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: combined-pod
spec:
  containers:
  - name: test-container
    image: busybox
    command: ["/bin/sh", "-c", "sleep 3600"]

    # Inject ConfigMap as Environment Variables
    envFrom:
    - configMapRef:
        name: final-config

    # Mount Secret as Files
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secret
      readOnly: true

  volumes:
  - name: secret-volume
    secret:
      secretName: db-secret-yaml

ubuntu@master:~$
```

Step 5: Deploy the Pod

```
kubectl apply -f combined-pod.yaml
```

Check status:

```
kubectl get pods
```

Wait until STATUS = Running.

```
secretName: ab-secret-yaml

ubuntu@master:~$ kubectl apply -f combined-pod.yaml
pod/combined-pod created
ubuntu@master:~$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
combined-pod        1/1     Running   0           11s
```

Step 3: Verify ConfigMap (Environment Variables)

```
kubectl exec -it combined-pod -- printenv | grep APP
```

```
ubuntu@master:~$
ubuntu@master:~$ kubectl exec -it combined-pod -- printenv | grep APP
APP_ENV=Production
APP_NAME=FinalApp
APP_VERSION=1.0
```

Step 4: Verify Secret (Mounted as Files)

List files:

```
kubectl exec -it combined-pod -- ls /etc/secret
```

Expected:

```
username
password
```

Check values:

```
kubectl exec -it combined-pod -- cat /etc/secret/username
kubectl exec -it combined-pod -- cat /etc/secret/password
```

Expected:

```
waseem
MyStrongPass123
```



```
ubuntu@master:~$ kubectl exec -it combined-pod -- ls /etc/secret
password username
ubuntu@master:~$ kubectl exec -it combined-pod -- cat /etc/secret/username
waseemubuntu@master:~$ kubectl exec -it combined-pod -- cat /etc/secret/password
MyStrongPass123ubuntu@master:~$
```

Final Result

Inside the same Pod:

- ✓ ConfigMap → Environment Variables
- ✓ Secret → Files inside `/etc/secret`
- ✓ Both accessible successfully

